

DEEPSEEK

```
import numpy as np
import matplotlib.pyplot as plt
import random
from datetime import datetime
```

```
# Configuração para gráficos em português
plt.rcParams['axes.unicode_minus'] = False
```

```
def validar_entrada_positiva(mensagem, permitir_zero=False):
    """Valida entrada do usuário garantindo que seja um número positivo"""
    while True:
        try:
            valor = float(input(mensagem))
            if permitir_zero and valor >= 0:
                return valor
            elif not permitir_zero and valor > 0:
                return valor
            else:
                print("❌ Erro: Valor inválido. Digite um número positivo.")
        except ValueError:
            print("❌ Erro: Digite um número válido.")
```

```
def validar_percentual(mensagem):
    """Valida percentual entre 0 e 100"""
    while True:
        try:
            valor = float(input(mensagem))
            if 0 <= valor <= 100:
                return valor
            else:
                print("❌ Erro: Percentual deve estar entre 0 e 100.")
        except ValueError:
            print("❌ Erro: Digite um número válido.")
```

```
def validar_volume_inicial(mensagem, capacidade):
    """Valida volume inicial não excedendo a capacidade"""
    while True:
        try:
            volume = float(input(mensagem))
            if 0 <= volume <= capacidade:
                return volume
            else:
                print(f"❌ Erro: Volume deve estar entre 0 e {capacidade} m³")
        except ValueError:
            print("❌ Erro: Digite um número válido.")
```

```

def calcular_indicadores(demanda, perda_percentual, capacidade, volume_inicial,
custo_bombeamento):
    """Calcula todos os indicadores para um bairro"""
    volume_perdido = demanda * (perda_percentual / 100)
    volume_entregue = demanda - volume_perdido
    custo_diario = demanda * custo_bombeamento # Custo baseado no volume bombeado

    # Autonomia em dias (considerando volume inicial e demanda diária)
    if demanda > 0:
        autonomia = volume_inicial / demanda
    else:
        autonomia = float('inf')

    # Classificação de risco
    if autonomia > 3:
        risco = "Baixo"
    elif autonomia >= 1:
        risco = "Médio"
    else:
        risco = "Alto"

    return {
        'volume_entregue': volume_entregue,
        'volume_perdido': volume_perdido,
        'custo_diario': custo_diario,
        'autonomia': autonomia,
        'risco': risco
    }

def simular_7_dias(bairros_dados):
    """
    Simula 7 dias com demanda variando ±20%
    Retorna: historico_volumes, dias_criticos, historico_demandas
    """

    # Inicializar históricos
    historico_volumes = {bairro['nome']: [bairro['volume_inicial']] for bairro in bairros_dados}
    historico_demandas = {bairro['nome']: [] for bairro in bairros_dados}
    dias_criticos = {bairro['nome']: [] for bairro in bairros_dados}

    # Criar cópia dos volumes atuais
    volumes_atuais = {bairro['nome']: bairro['volume_inicial'] for bairro in bairros_dados}

    # Simular 7 dias
    for dia in range(1, 8):
        for bairro in bairros_dados:
            nome = bairro['nome']
            demanda_base = bairro['demanda']

```

```

# Variação aleatória de ±20%
variacao = random.uniform(-0.2, 0.2)
demanda_dia = demanda_base * (1 + variacao)
demanda_dia = max(0, demanda_dia) # Não permitir demanda negativa

historico_demandas[nome].append(demanda_dia)

# Atualizar volume
volume_anterior = volumes_atuais[nome]
novo_volume = volume_anterior - demanda_dia

# Verificar situação crítica (volume negativo ou zerou)
if novo_volume < 0:
    dias_criticos[nome].append(dia)
    novo_volume = 0

# Atualizar volumes
volumes_atuais[nome] = novo_volume
historico_volumes[nome].append(novo_volume)

return historico_volumes, historico_demandas, dias_criticos

def exibir_tabela_resultados(resultados, demandas_originais):
    """SAÍDA 1: Exibe tabela formatada com os resultados por bairro"""
    print("\n" + "="*100)
    print("🏡 TABELA DE RESULTADOS POR BAIRRO")
    print("="*100)

    # Cabeçalho
    print(f"\n{'Bairro':<20} {'Demanda':<12} {'Vol. Entregue':<15} {'Vol. Perdido':<15} {'Custo (R$)':<15} {'Autonomia':<12} {'Risco':<10}")
    print("-"*99)

    # Linhas de dados
    for i, r in enumerate(resultados):
        print(f"{r['nome']:<20} {demandas_originais[i]:<12.2f} {r['volume_entregue']:<15.2f} {r['volume_perdido']:<15.2f} "
              f"{r['custo_diario']:<15.2f} {r['autonomia']:<12.2f} {r['risco']:<10}")

    print("-"*99)

def identificar_bairro_critico(resultados):
    """SAÍDA: Identifica automaticamente o bairro mais crítico"""
    # Filtrar bairros com demanda positiva
    resultados_validos = [r for r in resultados if r['autonomia'] != float('inf')]

    if not resultados_validos:
        return None

```

```
bairro_critico = min(resultados_validos, key=lambda x: x['autonomia'])
```

```
print("\n" + "="*60)
print("⚠️ BAIRRO MAIS CRÍTICO IDENTIFICADO")
print("="*60)
print(f"📍 Nome: {bairro_critico['nome']}")
print(f"📅 Autonomia: {bairro_critico['autonomia']:.2f} dias")
print(f"🔥 Nível de Risco: {bairro_critico['risco']}")
if bairro_critico['autonomia'] < 1:
    print("🔴 ALERTA: Risco ALTO de desabastecimento imediato!")
elif bairro_critico['autonomia'] <= 3:
    print("🟡 ATENÇÃO: Risco MÉDIO de desabastecimento")
else:
    print("🟢 OK: Situação confortável")
print("="*60)
```

```
return bairro_critico
```

```
def grafico_barras_demanda_vs_entregue(nomes, demandas, volumes_entregues):
```

```
    """SAÍDA 2: Gráfico de barras comparando demanda e volume entregue"""
```

```
    fig, ax = plt.subplots(figsize=(12, 7))
```

```
    x = np.arange(len(nomes))
```

```
    width = 0.35
```

```
    barras1 = ax.bar(x - width/2, demandas, width, label='Demanda (m³/dia)',
                     color='#3498db', alpha=0.8, edgecolor='black', linewidth=1)
```

```
    barras2 = ax.bar(x + width/2, volumes_entregues, width, label='Volume Entregue (m³/dia)',
                     color='#2ecc71', alpha=0.8, edgecolor='black', linewidth=1)
```

```
    # Adicionar valores nas barras
```

```
    for barra in barras1:
```

```
        altura = barra.get_height()
```

```
        ax.text(barra.get_x() + barra.get_width()/2., altura + max(demandas)*0.02,
                 f'{altura:.0f}', ha='center', va='bottom', fontsize=10, fontweight='bold')
```

```
    for barra in barras2:
```

```
        altura = barra.get_height()
```

```
        ax.text(barra.get_x() + barra.get_width()/2., altura + max(demandas)*0.02,
                 f'{altura:.0f}', ha='center', va='bottom', fontsize=10, fontweight='bold')
```

```
    ax.set_xlabel('Bairros', fontsize=13, fontweight='bold')
```

```
    ax.set_ylabel('Volume (m³/dia)', fontsize=13, fontweight='bold')
```

```
    ax.set_title('Comparação: Demanda vs Volume Efetivamente Entregue', fontsize=14,
                 fontweight='bold', pad=20)
```

```
    ax.set_xticks(x)
```

```
    ax.set_xticklabels(nomes, rotation=45, ha='right')
```

```

ax.legend(loc='upper right', fontsize=12, framealpha=0.9)
ax.grid(True, alpha=0.3, linestyle='--', axis='y')

# Adicionar anotação da perda
for i, (d, ve) in enumerate(zip(demandas, volumes_entregues)):
    perda_percentual = ((d - ve) / d * 100) if d > 0 else 0
    ax.annotate(f'Perda: {perda_percentual:.1f}%',
                xy=(i, ve + max(demandas)*0.01),
                fontsize=8, ha='center', style='italic')

plt.tight_layout()
plt.show()

def grafico_barras_custo(nomes, custos):
    """SAÍDA 3: Gráfico de barras com o custo diário por bairro"""
    fig, ax = plt.subplots(figsize=(12, 7))

    # Cores gradiente (verde para menor custo, vermelho para maior)
    if max(custos) > 0:
        cores = plt.cm.RdYlGn_r(np.array(custos) / max(custos))
    else:
        cores = ['#95a5a6'] * len(custos)

    barras = ax.bar(nomes, custos, color=cores, edgecolor='black', linewidth=1.5)

    # Adicionar valores nas barras
    for barra, custo in zip(barras, custos):
        ax.text(barra.get_x() + barra.get_width()/2., barra.get_height() + max(custos)*0.02,
                f'R$ {custo:,.2f}', ha='center', va='bottom', fontsize=11, fontweight='bold')

    ax.set_xlabel('Bairros', fontsize=13, fontweight='bold')
    ax.set_ylabel('Custo Diário (R$)', fontsize=13, fontweight='bold')
    ax.set_title('Custo Diário de Bombeamento por Bairro', fontsize=14, fontweight='bold',
pad=20)
    ax.tick_params(axis='x', rotation=45)
    ax.grid(True, alpha=0.3, axis='y', linestyle='--')

    # Adicionar linha de média
    media_custo = np.mean(custos)
    ax.axhline(y=media_custo, color='red', linestyle='--', linewidth=2, alpha=0.7, label=f'Média:
R$ {media_custo:,.2f}')
    ax.legend(loc='upper right', fontsize=10)

    plt.tight_layout()
    plt.show()

def grafico_pizza_riscos(resultados):
    """SAÍDA 4: Gráfico de pizza mostrando a distribuição dos níveis de risco"""

```

```

riscos = [r['risco'] for r in resultados]
categorias = ['Baixo', 'Médio', 'Alto']
contagem = [riscos.count(cat) for cat in categorias]
cores = ['#2ecc71', '#f39c12', '#e74c3c']

# Filtrar apenas categorias com valores > 0
dados_validos = [(cat, cont, cores[i]) for i, (cat, cont) in enumerate(zip(categorias,
contagem)) if cont > 0]

if not dados_validos:
    print(" ⚠ Sem dados para gerar gráfico de pizza")
    return

labels = [d[0] for d in dados_validos]
sizes = [d[1] for d in dados_validos]
colors_pie = [d[2] for d in dados_validos]
explode = [0.05] * len(labels)

fig, ax = plt.subplots(figsize=(10, 8))

# Criar gráfico de pizza
wedges, texts, autotexts = ax.pie(sizes, labels=labels, autopct='%1.1f%%',
                                colors=colors_pie, explode=explode,
                                startangle=90, shadow=True,
                                textprops={'fontsize': 13})

# Personalizar textos
for autotext in autotexts:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
    autotext.set_fontsize(14)

for text in texts:
    text.set_fontweight('bold')
    text.set_fontsize(12)

ax.set_title('Distribuição dos Níveis de Risco de Desabastecimento',
            fontsize=15, fontweight='bold', pad=25)

# Adicionar legenda detalhada
legenda_texto = []
for cat, cont in zip(categorias, contagem):
    if cont > 0:
        legenda_texto.append(f'{cat}: {cont} bairro(s)')

if legenda_texto:
    ax.legend(legenda_texto, loc='upper right', bbox_to_anchor=(1.3, 1), fontsize=11)

```

```

plt.tight_layout()
plt.show()

def grafico_linha_evolucao_7dias(historico_volumes, dias_criticos):
    """DESAFIO: Gráfico de linha mostrando evolução do volume em 7 dias"""
    fig, ax = plt.subplots(figsize=(14, 8))

    dias = list(range(8)) # Dias 0 a 7
    cores_linhas = ['#2c3e50', '#16a085', '#c0392b', '#2980b9', '#8e44ad', '#d35400',
                    '#27ae60', '#e67e22']

    for i, (nome, volumes) in enumerate(historico_volumes.items()):
        cor = cores_linhas[i % len(cores_linhas)]
        ax.plot(dias, volumes, marker='o', linewidth=2.5, markersize=8,
                label=nome, color=cor, markeredgewidth=1.5)

    # Destacar pontos críticos (onde zerou)
    for j, vol in enumerate(volumes):
        if vol == 0 and j > 0 and j in dias_criticos.get(nome, []):
            ax.plot(j, 0, 'ro', markersize=14, markeredgewidth=2, zorder=5)
            ax.annotate(f'Crítico!\nDia {j}', xy=(j, 0), xytext=(j, max(volumes)*0.1),
                        arrowprops=dict(arrowstyle='->', color='red', lw=1.5),
                        fontsize=9, ha='center', color='red', fontweight='bold')

    ax.set_xlabel('Dias', fontsize=13, fontweight='bold')
    ax.set_ylabel('Volume no Reservatório (m³)', fontsize=13, fontweight='bold')
    ax.set_title('Evolução do Volume nos Reservatórios ao Longo de 7 Dias',
                fontsize=15, fontweight='bold', pad=20)
    ax.set_xticks(dias)
    ax.set_xticklabels(['Dia 0\n(Inicial)', 'Dia 1', 'Dia 2', 'Dia 3',
                        'Dia 4', 'Dia 5', 'Dia 6', 'Dia 7'])
    ax.grid(True, alpha=0.3, linestyle='--')
    ax.legend(loc='best', fontsize=11, framealpha=0.9)

    # Linha de limite crítico
    ax.axhline(y=0, color='red', linestyle='--', linewidth=2, alpha=0.7, label='Limite Crítico')

    # Preencher área abaixo de zero (se houver)
    ax.fill_between(dias, -max(volumes)*0.1, 0, alpha=0.2, color='red', label='Zona de Risco')

    plt.tight_layout()
    plt.show()

def exibir_simulacao_dias_criticos(dias_criticos):
    """Exibe em quais dias cada bairro entrou em situação crítica"""
    print("\n" + "="*60)
    print("📊 RESULTADO DA SIMULAÇÃO DE 7 DIAS")

```

```
print("="*60)
```

```
teve_critico = False
```

```
for bairro, dias in dias_criticos.items():
```

```
    if dias:
```

```
        print(f"\n❌ {bairro}:")
```

```
        print(f"    Dias críticos: {' '.join(map(str, dias))}")
```

```
        print(f"    ⚠️ Entrou em situação crítica {len(dias)} vez(es)")
```

```
        teve_critico = True
```

```
    else:
```

```
        print(f"\n✅ {bairro}: Nenhum dia crítico")
```

```
if not teve_critico:
```

```
    print(f"\n✅ Nenhum bairro atingiu situação crítica durante toda a simulação!")
```

```
print("="*60)
```

```
def main():
```

```
    """Função principal do programa"""
```

```
    print("="*60)
```

```
    print(f"🏠 SISTEMA DE ANÁLISE DE ABASTECIMENTO DE ÁGUA")
```

```
    print("="*60)
```

```
    print("\nEste programa analisa o desempenho do sistema de abastecimento")
```

```
    print("e gera relatórios completos com gráficos e simulações.\n")
```

```
# Entrada do número de bairros
```

```
while True:
```

```
    try:
```

```
        n_bairros = int(input("🏠 Número de bairros a analisar: "))
```

```
        if n_bairros > 0:
```

```
            break
```

```
        print(f"❌ Erro: Digite um número positivo.")
```

```
    except ValueError:
```

```
        print(f"❌ Erro: Digite um número inteiro válido.")
```

```
bairros = []
```

```
# Coleta de dados por bairro
```

```
for i in range(n_bairros):
```

```
    print(f"\n{'='*50}")
```

```
    print(f"🏠 BAIRRO {i+1}")
```

```
    print(f"{'='*50}")
```

```
    nome = input("📄 Nome do bairro: ").strip()
```

```
    while not nome:
```

```
        nome = input("📄 Nome não pode estar vazio. Digite novamente: ").strip()
```

```
    demanda = validar_entrada_positiva("💧 Demanda diária (m³/dia): ")
```



```

capacidade = validar_entrada_positiva(" 🏠 Capacidade do reservatório (m³): ")

volume_inicial = validar_volume_inicial(f" 🏠 Volume inicial (m³) [max: {capacidade}]: ",
capacidade)

perda = validar_percentual(" 🧰 Percentual de perda na tubulação (%): ")
custo_bombeamento = validar_entrada_positiva(" 💰 Custo de bombeamento (R$/m³):
")

bairros.append({
    'nome': nome,
    'demanda': demanda,
    'capacidade': capacidade,
    'volume_inicial': volume_inicial,
    'perda': perda,
    'custo_bombeamento': custo_bombeamento
})

# Processamento dos indicadores
resultados = []
demandas_originais = []
volumes_entregues_lista = []
custos_lista = []

for bairro in bairros:
    indicadores = calcular_indicadores(
        bairro['demanda'],
        bairro['perda'],
        bairro['capacidade'],
        bairro['volume_inicial'],
        bairro['custo_bombeamento']
    )

    resultados.append({
        'nome': bairro['nome'],
        **indicadores
    })

    demandas_originais.append(bairro['demanda'])
    volumes_entregues_lista.append(indicadores['volume_entregue'])
    custos_lista.append(indicadores['custo_diario'])

# SAÍDA 1: Tabela de resultados
exibir_tabela_resultados(resultados, demandas_originais)

# SAÍDA: Bairro mais crítico
bairro_critico = identificar_bairro_critico(resultados)

```

```

# DESAFIO: Simulação de 7 dias
print("\n" + "="*60)
print("🔄 INICIANDO SIMULAÇÃO DE 7 DIAS")
print(" Demanda variará aleatoriamente ±20% por dia")
print("="*60)

# Criar cópia para simulação
bairros_simulacao = [b.copy() for b in bairros]
historico_volumes, historico_demandas, dias_criticos =
simular_7_dias(bairros_simulacao)

# Exibir dias críticos da simulação
exibir_simulacao_dias_criticos(dias_criticos)

# GRÁFICOS
print("\n" + "="*60)
print("📊 GERANDO GRÁFICOS...")
print("="*60)

# SAÍDA 2: Gráfico demanda vs volume entregue
grafico_barras_demanda_vs_entregue([r['nome'] for r in resultados],
                                   demandas_originais,
                                   volumes_entregues_lista)

# SAÍDA 3: Gráfico de custo diário
grafico_barras_custo([r['nome'] for r in resultados], custos_lista)

# SAÍDA 4: Gráfico de pizza dos riscos
grafico_pizza_riscos(resultados)

# DESAFIO: Gráfico de linha da evolução dos volumes
grafico_linha_evolucao_7dias(historico_volumes, dias_criticos)

# Resumo final
print("\n" + "="*60)
print("✅ ANÁLISE CONCLUÍDA COM SUCESSO!")
print("="*60)
print("Foram gerados:")
print("📄 1. Tabela de resultados por bairro")
print("📊 2. Gráfico de barras (Demanda vs Volume Entregue)")
print("💰 3. Gráfico de barras (Custo diário)")
print("🍕 4. Gráfico de pizza (Distribuição de riscos)")
print("📈 5. Gráfico de linha (Evolução dos volumes em 7 dias)")
print(f"\n🏆 Bairro mais crítico: {bairro_critico['nome'] if bairro_critico else 'N/A'}")
print("="*60)

if __name__ == "__main__":
    random.seed(42) # Para reprodutibilidade

```

main()